

Counting Sort

O Counting Sort ordena inteiros contando quantas vezes cada valor aparece. Em vez de comparar elementos, ele usa um vetor auxiliar de contagem.

Funcionamento

Considerando o vetor [4, 2, 2, 8, 3, 3, 1]: 1. Encontrar o maior valor do vetor. 2. Criar um vetor contagem com posições de 0 até o maior valor. 3. Para cada elemento, incrementar contagem[elemento]. 4. Percorrer o vetor de contagem e reconstruir o vetor original na ordem correta.

Vetor original=>Contar ocorrências de cada valor=>Percorrer as contagens em ordem crescente=>Vetor ordenado.

O Count Sort é mais eficiente quando o intervalo de valores é pequeno em relação à quantidade de elementos. A implementação abaixo considera apenas inteiros não negativos.

Complexidade do Count Sort

Se n é a quantidade de elementos e k é o maior valor do vetor, o custo é $O(n + k)$.

Característica Count Sort

Método Não comparação. Melhor $O(n + k)$. Médio $O(n + k)$. Pior $O(n + k)$. Memória $O(k)$. Estável? Não, nesta versão. In-place? Não

Radix Sort LSD (Least Significant Digit)

O Radix Sort LSD (Least Significant Digit) ordena inteiros analisando seus dígitos do menos significativo para o mais significativo. Portanto, começa pela casa das unidades, depois passa para as dezenas, centenas e assim por diante. Em cada posição decimal, ele utiliza uma ordenação estável por contagem.

Funcionamento

No Radix Sort LSD, o algoritmo: 1. Identifica o maior número para descobrir quantas casas decimais serão processadas. 2. Ordena os elementos pelo dígito das unidades usando uma ordenação estável. 3. Repete o processo para as dezenas, centenas e demais casas decimais. 4. Mantém a ordem relativa dos elementos que possuem o mesmo dígito. 5. Para quando todas as casas do maior número forem processadas. Exemplo para o vetor [170, 45, 75, 90, 802, 24, 2, 66]:

Unidades → 170, 90, 802, 2, 24, 45, 75, 66

Dezenas → 802, 2, 24, 45, 66, 170, 75, 90

Centenas → 2, 24, 45, 66, 75, 90, 170, 802

Essa é a estratégia LSD, pois a ordenação começa pelo dígito menos significativo. O Radix Sort LSD abaixo usa base decimal e considera inteiros não negativos. A estabilidade da ordenação por cada dígito é necessária para preservar o resultado das casas já processadas.

Complexidade do Radix Sort LSD

Se n é a quantidade de elementos, d é a quantidade de dígitos do maior valor e b é a base utilizada, a complexidade é $O(d(n + b))$. Com base decimal fixa, $b = 10$, então pode ser escrita como $O(dn)$.

Característica Radix Sort LSD

Método Não comparação. Melhor $O(d(n + b))$. Médio $O(d(n + b))$. Pior $O(d(n + b))$. Memória $O(n + b)$. Estável? Sim. In-place? Não

```
void countSort(int v[], int n) {
    if (n <= 0)
        return;

    int maior = v[0];
    for (int i = 1; i < n; i++) {
        if (v[i] > maior)
            maior = v[i];
    }

    int* contagem = new int[maior + 1];
    for (int i = 0; i <= maior; i++)
        contagem[i] = 0;

    for (int i = 0; i < n; i++)
        contagem[v[i]]++;

    int posicao = 0;
    for (int valor = 0; valor <= maior; valor++) {
        while (contagem[valor] > 0) {
            v[posicao] = valor;
            posicao++;
            contagem[valor]--;
        }
    }

    delete[] contagem;
}
```

```
int maiorValor(int v[], int n) {
    int maior = v[0];
    for (int i = 1; i < n; i++) {
        if (v[i] > maior)
            maior = v[i];
    }
    return maior;
}

void ordenarPorDigito(int v[], int n, int divisor) {
    int* saida = new int[n];
    int contagem[10];
    for (int i = 0; i < 10; i++)
        contagem[i] = 0;

    for (int i = 0; i < n; i++) {
        int digito = (v[i] / divisor) % 10;
        contagem[digito]++;
    }

    for (int i = 1; i < 10; i++)
        contagem[i] += contagem[i - 1];

    // Percorrer de trás para frente mantém a ordenação estável.
    for (int i = n - 1; i >= 0; i--) {
        int digito = (v[i] / divisor) % 10;
        saida[contagem[digito] - 1] = v[i];
        contagem[digito]--;
    }

    for (int i = 0; i < n; i++)
        v[i] = saida[i];
    delete[] saida;
}

void radixSort(int v[], int n) {
    if (n <= 0)
        return;

    int maior = maiorValor(v, n);

    for (int divisor = 1; maior / divisor > 0; divisor *= 10)
        ordenarPorDigito(v, n, divisor);
}
```

Algoritmos de Ordenação Estáveis e Instáveis

A estabilidade de um algoritmo de ordenação está relacionada à forma como ele trata elementos que possuem a mesma chave de ordenação.

Ordenação Estável

Um algoritmo de ordenação é considerado estável quando mantém a ordem relativa original entre elementos que possuem a mesma chave de ordenação.

Exemplos de algoritmos estáveis:

Bubble Sort, Insertion Sort, Merge Sort, Counting Sort (quando implementado de forma estável) e Radix Sort (quando utiliza uma ordenação estável internamente)

Ordenação Instável

Um algoritmo de ordenação é considerado instável quando pode alterar a ordem relativa entre elementos que possuem a mesma chave de ordenação.

Exemplos de algoritmos instáveis:

Selection Sort, Quick Sort (na implementação tradicional), Heap Sort

Exemplo de Estabilidade

Considere uma lista de alunos que será ordenada pela nota:

Ordem original Aluno Nota

1 João 8/ 2 Maria 7/ 3 Pedro 8/ 4 Ana 9

Ordenação estável

Após a ordenação pela nota:

Ordem Aluno Nota

1 Maria 7/ 2 João 8/ 3 Pedro 8/ 4 Ana 9. João continua antes de Pedro, pois ambos possuem nota 8 e João estava originalmente antes de Pedro.

Ordenação instável

Em uma ordenação instável, a ordem relativa dos elementos com a mesma chave pode ser alterada:

Ordem Aluno Nota

1 Maria 7/ 2 Pedro 8/ 3 João 8/ 4 Ana 9. A lista continua corretamente ordenada pelas notas, porém Pedro passou a aparecer antes de João.